Projeto Integrador: Fábrica Inteligente – MetalTech Soluções Industriais S.A.

Ferramentas Necessárias: Python (Jupyter Notebook / Google Colab), Pandas, NumPy, Scikit-Learn, OpenCV, Flask/FastAPI, Matplotlib/Seaborn.

Contextualização

A MetalTech Soluções Industriais S.A. instalou recentemente sensores IoT nas suas linhas de produção. Contudo, os dados gerados (temperatura, vibração, pressão) não estão a ser aproveitados. A fábrica enfrenta paradas não planeadas por quebra de motores e defeitos na produção que passam despercebidos pela inspeção visual humana. Vocês foram encarregues de desenvolver o protótipo do projeto "Fábrica Inteligente: Preditiva e Visão Computacional" em 3 etapas.

Etapa 1: Diagnóstico e Tratamento de Dados Industriais (22h)

Cenário: O sistema de IoT da fábrica exportou uma amostra de 10.000 registos (dados_sensores_metaltech.csv). Os sensores sofreram interferências eletromagnéticas, gerando ruídos de medição, valores absurdos (outliers) e perdas de pacotes (dados nulos).

Missão do Aluno:

1. **Importação e Inspeção:** Carregar o ficheiro CSV utilizando a biblioteca pandas. Identificar o tipo de dados de cada coluna e quantificar os valores ausentes.
2. **Limpeza de Dados (Data Cleaning):**
    - Tratar os valores nulos (NaN) utilizando técnicas adequadas para séries temporais (ex: interpolate ou forward fill).
    - Identificar e remover/tratar os "outliers" (ex: temperaturas de 999.9 e vibrações negativas) com base na lógica física operacional ou usando o método de *Z-score*.
3. **Análise Exploratória (EDA):**
    - Gerar gráficos (histogramas, boxplots) das variáveis limpas.
    - Criar uma matriz de correlação para identificar a relação entre o aumento de temperatura/vibração e a variável alvo falha_motor.
4. **Entregável:** Script Jupyter Notebook documentado e um breve relatório (PDF/Markdown) com os principais achados da análise de dados.

Etapa 2: Inteligência Preditiva, Treinamento

Cenário: Com os dados limpos, a fábrica precisa agora de um modelo capaz de antecipar as falhas nos motores, alertando os operadores antes que a máquina pare.

Missão do Aluno:

1. **Preparação para Machine Learning:** Separar o dataset em variáveis previsoras (X) e a variável alvo (y = falha_motor). Dividir os dados em conjuntos de Treino (70%) e Teste (30%). Realizar o escalonamento numérico (ex: StandardScaler).
2. **Treinamento de Modelos:**
    - Treinar, no mínimo, dois algoritmos de classificação diferentes (ex: *Random Forest*, *SVM* ou *Regressão Logística*).
    - Aplicar um algoritmo de agrupamento (ex: *K-Means*) para tentar identificar, sem os rótulos de falha, se o modelo consegue separar os "estados operacionais" (normal vs. crítico).
3. **Avaliação:** Comparar os modelos utilizando a Matriz de Confusão e métricas industriais (Acurácia, Precisão, Recall e F1-Score). *Dica: Focar no Recall é vital, pois a fábrica não pode deixar uma falha passar despercebida.*
4. **Entregável:** O código de treino salvo, o ficheiro do modelo exportado (.pkl ou .joblib), uma tabela comparativa de resultados e o rascunho de um plano simples de MLOps (como a equipa iria monitorizar a degradação deste modelo ao longo de 6 meses).

Etapa 3: Visão Computacional, APIs e Análise Socioética (22h)

Cenário: O modelo preditivo está pronto, mas precisa ser acedido pelos CLPs da fábrica. Paralelamente, precisamos substituir a falha humana na inspeção de qualidade visual das peças utilizando câmaras.

Missão do Aluno:

1. **Visão Computacional (OpenCV):**
    - Criar um script em Python utilizando OpenCV para analisar imagens estáticas ou um vídeo curto (o professor pode fornecer imagens de peças perfeitas e peças com defeitos, como parafusos enferrujados ou engrenagens rachadas).
    - Aplicar filtros (escala de cinza, *blur*, *Canny Edge Detection*) para identificar os contornos e acusar falha se a geometria da peça não corresponder ao padrão esperado.
2. **Integração (API RESTful):**
    - Desenvolver uma API simples utilizando Flask ou FastAPI. A API deve ter uma rota (/predict) que receba os dados de um sensor via JSON (temperatura, vibração, pressão) e retorne a previsão do modelo treinado na Etapa 2 ("Risco de Falha: Sim/Não").
3. **Reflexão Ética e Regulatória:**
    - Redigir um parecer técnico de 2 páginas abordando:
        - O impacto sociolaboral desta automação (o que acontece com os inspetores de qualidade?).
        - Questões de cibersegurança do modelo na rede industrial.
        - Como a LGPD se aplica se câmaras de visão computacional na fábrica acabarem por captar o rosto dos operadores durante o turno.
4. **Entregáveis:** Script do OpenCV funcional, o código da API (testado via *Postman* ou *Insomnia*) e o Parecer Técnico/Ético em PDF.